



SafeCross

AI-Based Smart Crosswalk System

Specification & Scientific Basis Document

Initiation document for implementation (SRS / PRD)

Student name	ID number
Netanel Kadoshi	318829041
Or Smolarchik	208635045
Rachel Brodsky	212781835
Yosef Yosef	211364740
Liel Babayn	323834994

Item	Details
Mentor / Moderator	Netanel Ben Hamo
Institution	Center of Projects and Industrial Cooperation, Faculty of Science, HIT – Holon Institute of Technology
Date	04/10/2026
GitHub repository	SafeCross
YouTube demo video	[YouTube link – to be added]

Table of Contents

0. Administration	4
0.1 Introduction – Existing Situation (Problem Definition)	4
0.2 Rationale for Developing the System.....	4
0.3 Project Goals	4
0.4 Concepts and Definitions	5
1. System Description	6
1.1 The Character of the Application	6
1.2 System Users	6
1.3 Usage Scenarios (Use Cases).....	6
1.3.1 Login (all users)	6
1.3.2 “Admin” use cases	6
1.3.3 “Dispatcher” use cases	7
1.3.4 “Manager” use cases	7
1.3.5 “Technician” use cases.....	7
1.3.6 AI sensor use case	7
2. Functional Requirements	9
3. Non-Functional Requirements	10
3.1 Performance	10
3.2 Information Security	10
3.3 Scalability	10
3.4 Reliability	10
3.5 Usability and Maintainability.....	10
4. System Architecture and Design	11
4.1 Architecture Overview	11
4.2 Server-Side Architecture (Backend Modules).....	12
4.2.1 REST API	12
4.2.2 Real-Time Events (Socket.io).....	13
4.3 Client-Side Architecture	13
4.4 AI Service Pipeline	13
5. Data Structure (Database).....	14
5.1 Collections	14
6. Scientific Basis, Technologies and Algorithms	15
6.1 Object Detection – Convolutional Neural Networks (YOLOv8)	15
6.2 Multi-Object Tracking	15
6.3 Geometry – Distance to the Curb	15
6.4 Kinematic Risk Classification (Rule-Based Expert System)	15
6.5 Data Science – Server-Side Aggregation	16
6.6 Information Security	16
6.7 Event-Driven and Real-Time Communication	16
6.8 Technology Stack.....	17

7. User Interface (UI)	18
7.1 Admin Dashboard – Users and Hardware Infrastructure.....	18
7.2 Manager Dashboard – Safety Analytics.....	18
7.3 Dispatcher Dashboard – Real-Time Control Center	19
7.4 Technician Dashboard – Field Maintenance	19
8. Work Flow	20
9. Testing and Validation	21
9.1 Backend End-to-End Tests	21
9.2 AI Service Unit Tests.....	21
9.3 Manual and Integration Testing.....	21
10. Solution Overview and Implementation	22
10.1 Main Challenges and Solutions	22
10.1.1 Evolving requirements.....	22
10.1.2 Multi-role real-time synchronization.....	22
10.1.3 Reliable statistics.....	22
10.1.4 Camera-independent AI rules	22
10.1.5 Schema expansion without breaking data	23
11. Recommendations for Further Development.....	24
Links	24

Right-click the table and choose "Update Field" to refresh page numbers after editing.

0. Administration

0.1 Introduction – Existing Situation (Problem Definition)

Pedestrian crossings and intersections account for a disproportionate share of urban traffic accidents and near-misses. The most vulnerable road users – children, distracted pedestrians looking at a phone, and people crossing quickly or unexpectedly – are exactly the ones a driver is least likely to anticipate.

Today, monitoring of crosswalks relies on three approaches, each with a clear gap:

- **Traditional CCTV cameras** – passive recording that is used mainly for post-accident investigation. Nobody is warned while the dangerous situation is developing.
- **Passive (unsignalized) pedestrian crossings** – rely entirely on the driver's alertness. Drivers often fail to notice a pedestrian waiting at the curb, while pedestrians may step into the road unexpectedly.
- **Basic motion sensors** – react to any movement and therefore produce a high rate of false alarms, so operators learn to ignore them.

In addition, manual monitoring does not scale: a human operator cannot watch many locations continuously, and there is no single place where incidents, equipment health and statistics are collected for data-driven decisions.

0.2 Rationale for Developing the System

SafeCross closes these gaps by turning a regular camera at a crosswalk into an active safety sensor:

1. **Real-time detection** of pedestrians, children, vehicles and phones in the waiting area before the crosswalk, using a deep-learning object detector (YOLOv8).
2. **Behavioral risk classification** – not every crossing is an alert. Each tracked person is classified into one of 17 predefined risk cases (Low / Medium / High), so only unpredictable or dangerous behavior is escalated.
3. **Instant, prioritized alerts** are pushed to a control center (Dispatcher) in real time, with a snapshot of the event and its location on a map.
4. **Human verification** – the dispatcher confirms and handles each alert; only handled alerts enter the management statistics, which keeps the data reliable.
5. **Operational management** – infrastructure (junctions, cameras, LED warning strips), users and maintenance status are managed in one system with live synchronization across all connected screens.

0.3 Project Goals

- Build an end-to-end, real-time, multi-role web platform that combines computer vision (Python & YOLOv8), a REST + WebSocket backend (Node.js & Express, Socket.io) and a cloud database (MongoDB Atlas).
- Detect and classify dangerous and unpredictable pedestrian behavior automatically and deliver alerts to operators within seconds.
- Provide dedicated dashboards for four roles: Admin, Manager, Dispatcher and Technician.
- Compute safety statistics on the server (aggregation) to support data-driven decisions by municipalities.
- Prepare the system for activating a physical LED warning sign for drivers (each alert carries an `ledTriggered` flag).

0.4 Concepts and Definitions

Term	Definition
Junction / Crosswalk	A monitored pedestrian crossing. Stored with name, city, street, coordinates (lat/lng), status and a school-zone flag.
Edge zone (ROI)	A polygon (normalized 0–1 coordinates) marking the crosswalk / road area in the camera frame. Distances are measured from a person's feet to the curb line of this polygon.
Body-height unit (h)	The height of the person's bounding box. Distances and speeds are expressed in h and h/s, which makes the rules independent of camera distance and resolution.
Track	A sequence of detections of the same person across frames, with a stable ID.
Risk case	One of 17 predefined behaviors (e.g. H1 "bursting toward the road", M2 "approaching while holding a phone"), each mapped to a severity.
Severity	Low (logged only), Medium (early warning) or High (immediate danger).
ledTriggered	Flag on an alert: true for Medium/High alerts, meaning the alert should activate the LED warning sign.
Handled alert	An alert that a dispatcher has marked as resolved (<code>isResolved = true</code> , <code>resolvedAt</code> set by the server).
JWT	JSON Web Token – a signed token returned at login that identifies the user and role on every request.
API key (x-api-key)	A shared secret that authenticates the AI sensor service (machine-to-machine) instead of a user token.
Change stream	A MongoDB feature that notifies the server of every insert/update in a collection; used to broadcast alerts live.

1. System Description

1.1 The Character of the Application

- A web application (Single Page Application) built with React & Vite and styled with Tailwind CSS. It is responsive and works on desktop screens in the control center and on mobile devices in the field.
- A Node.js & Express backend exposes a RESTful API and a Socket.io WebSocket server on the same port.
- A separate Python AI service analyses video (YOLOv8 + tracking + risk rules) and sends alerts to the backend over HTTPS with an API key.
- Data is stored in MongoDB Atlas (cloud NoSQL) through Mongoose schemas; incident snapshots are stored in Cloudinary.
- An identification component (login) issues a JWT; every screen and API call is restricted by the user's role.
- Maps are displayed with Leaflet (React-Leaflet) and OpenStreetMap; addresses are converted to coordinates with the Nominatim geocoding service.

1.2 System Users

Role	Responsibility	Main capabilities
Admin	Primary system administrator	Create, edit, suspend and delete users; create, edit and delete junctions, cameras and LED strips (with automatic geocoding); sees all changes live.
Manager	Management tracking & analytics	Views server-computed statistics (KPIs, top-5 dangerous junctions, school zones, weekly trend, severity split); the dashboard refreshes live once the dispatcher marks the alert as resolved. Can export a PDF report.
Dispatcher	Real-time operational monitoring	Live alert table and interactive map; marks alerts as handled / re-opens them; changes are synchronized to every dispatcher on shift.
Technician	Field maintenance	Sees all equipment in "error" status marked by the admin, navigates to the junction and marks the equipment as fixed (active).
AI sensor (service)	Machine user	Posts hazard alerts with an API key; never uses a user token.

1.3 Usage Scenarios (Use Cases)

1.3.1 Login (all users)

- The user enters username and password. The server verifies the bcrypt hash and returns a JWT (valid 24 h) together with the role.
- The frontend routes the user to the matching dashboard: /admin, /manager, /dispatcher or /technician. A suspended user receives "Account is suspended".

1.3.2 "Admin" use cases

- **Manage users** – create a user with a role (register), update details or role, suspend / re-activate, delete. Every change is broadcast (`user_added`, `user_updated`, `user_deleted`) to all connected admins.
- **Manage hardware infrastructure** – add or edit a junction (name, status, city, street). The coordinates are filled automatically from the address (geocoding). Add cameras (IP, type:

LPR / PTZ / Thermal) and LED strips (color) and link them to a junction. Mark a junction as a school zone.

- **Live administrative synchronization** – after each create/update/delete the server emits `infra_added` / `infra_updated` / `infra_deleted` with `{ type, payload }`, so all admins see the same data without refreshing.

1.3.3 “Dispatcher” use cases

- On the main screen the dispatcher sees an interactive map of all junctions and a live alert table (severity, junction, time, person type, snapshot).
- A new alert appears instantly at the top of the table (`newAlert` event) without refreshing the page.
- The dispatcher reviews the event and clicks “Handled”. The server saves `isResolved = true` and `resolvedAt`, and broadcasts `alertUpdated` and `alert_resolved`. The status changes on every other dispatcher’s screen, preventing duplicated work. A handled alert can be re-opened (`alert_reopened`).
- From the map, the dispatcher can open a junction file (`/crosswalk/:id`) with its event history and equipment status.

1.3.4 “Manager” use cases

- The manager selects a filter: **Top 5** most dangerous junctions, **School zones**, or **All** junctions.
- The dashboard shows KPI cards (total handled alerts, high-risk alerts, active / total junctions, average response time), a bar chart per junction (children / adults / vehicles), a weekly line chart (Sunday–Saturday) and a severity pie chart.
- When any dispatcher handles an alert, the manager dashboard receives `alert_resolved` and re-fetches the statistics for the active filter automatically.
- The manager can export the dashboard as a PDF report with the “Export PDF report” button.

1.3.5 “Technician” use cases

- The technician sees a list of faulty equipment – cameras, LED strips and junctions whose status is “error” or “suspended” – with the junction location.
- After repairing, the technician marks the item as fixed. The status changes to “active” (PUT on the equipment) and the update is pushed live to admins, managers and dispatchers (single source of truth).

1.3.6 AI sensor use case

- The AI service analyses the camera video, detects a risky behaviour and sends `POST /api/alerts` with the `x-api-key` header, the severity, confidence, person type, distance / speed and an optional snapshot.
- The backend verifies the key, uploads the snapshot to Cloudinary, stores the alert in MongoDB and the change stream pushes it to the dispatchers.

1.4 User Navigation Flow

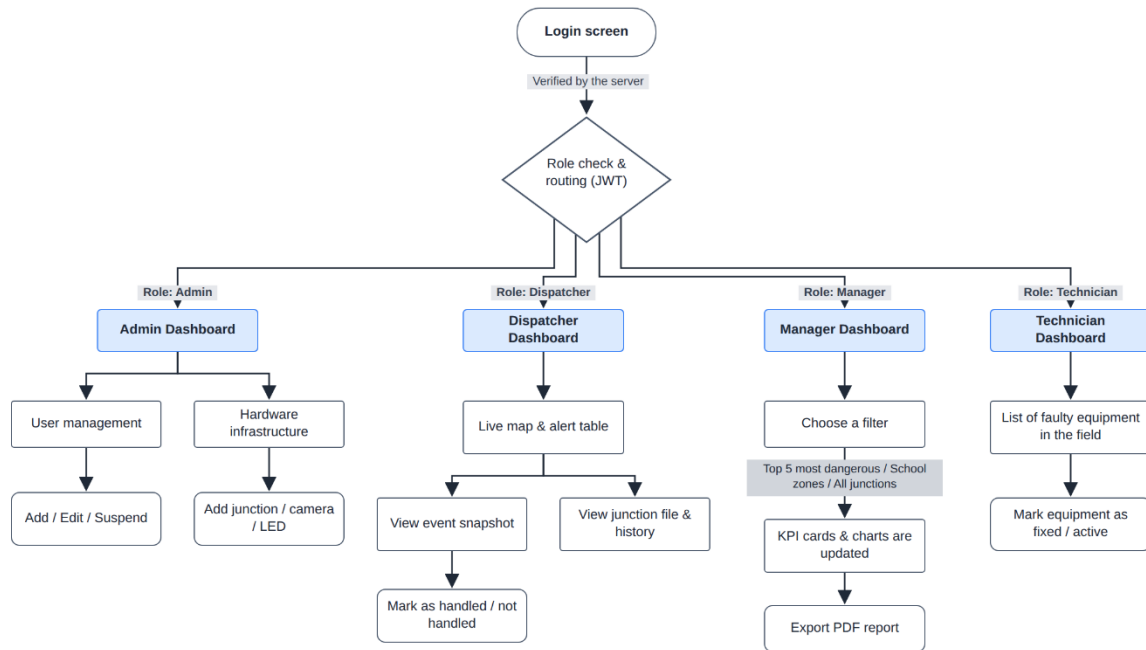


Figure 1 – Role-Based Navigation

2. Functional Requirements

Priority: **M** = Must, **S** = Should.

ID	Requirement	Role	Pri.
FR-01	Users log in with username and password and receive a JWT and their role.	All	M
FR-02	Each user is redirected to the dashboard of their role; other dashboards and APIs are blocked (403).	All	M
FR-03	Admin can create, update, suspend/activate and delete users. There is no public registration.	Admin	M
FR-04	Admin can create, update and delete junctions, cameras and LED strips; cameras and LEDs are linked to a junction.	Admin	M
FR-05	The system converts a junction's address to coordinates automatically (geocoding).	Admin	S
FR-06	Every infrastructure or user change is pushed live to all connected admins.	Admin	M
FR-07	The AI service detects persons, vehicles and phones in each analyzed video frame.	AI	M
FR-08	The AI service tracks each person across frames and classifies behavior into 17 risk cases with a severity and a confidence score.	AI	M
FR-09	The AI service sends one alert per incident to the backend, authenticated with an API key, together with a snapshot of the event. If the image upload fails, the alert is still saved without it.	AI	M
FR-10	New alerts appear on the dispatcher dashboard in real time, on a table and on a map.	Dispatcher	M
FR-11	The dispatcher can mark an alert as handled or re-open it; the server stores the resolution time, which is used to calculate the average response time on the manager dashboard.	Dispatcher	M
FR-12	The manager dashboard shows server-computed statistics filtered by Top 5 / School zones / All.	Manager	M
FR-13	Only handled alerts are counted in the statistics; the dashboard refreshes when an alert is handled.	Manager	M
FR-14	The technician sees all equipment in error/suspended status and can mark it as fixed.	Technician	M
FR-15	Each junction has a details page with its alert history and equipment status.	Dispatcher	S
FR-16	Alerts of Medium/High severity are flagged for LED warning activation (ledTriggered).	AI / System	S
FR-17	The manager can export the statistics dashboard as a PDF report.	Manager	S

3. Non-Functional Requirements

3.1 Performance

- An alert reaches the dispatcher screen within about one second after the backend receives it (MongoDB change stream → Socket.io push, no polling).
- The AI service analyses 1 of every N frames (default N = 3) on a reduced resolution (960 px width) so it can run on a regular CPU; network sending runs on a background thread and never blocks video analysis.
- Statistics are calculated in a single MongoDB aggregation (`$facet`) instead of loading all alerts into the server memory.

3.2 Information Security

- Passwords are stored only as bcrypt hashes; plain passwords are never saved or returned.
- Every user request carries `Authorization: Bearer <JWT>`; the token is signed with a server secret and expires after 24 hours.
- Role-based access control (RBAC) on every route – e.g. creating equipment is Admin-only, updating equipment is Admin + Technician.
- Machine endpoints (`POST /api/alerts`, `POST /api/detect`) are protected by an API key compared in constant time (protection against timing attacks); the server refuses to run them without a configured key (fail-closed).
- Secrets (database URI, JWT secret, API keys, Cloudinary credentials) are kept in environment variables (.env) and never committed to Git.
- Server-managed fields (such as `resolvedAt`) are ignored if sent by the client.

3.3 Scalability

- Stateless REST API – more server instances can be added behind a load balancer; the JWT holds the identity.
- MongoDB Atlas is a managed, replicated cloud database that can be scaled vertically or by sharding.
- The AI service is a separate process; one instance per camera can run in parallel, all reporting to the same API.
- Images are stored in Cloudinary (CDN), not in the database.

3.4 Reliability

- If the snapshot upload fails, the AI service re-sends the alert without the image, so a safety event is never lost.
- A socket emit failure never fails the HTTP request – the database write is the source of truth.
- Duplicate alerts are reduced by an incident window: one alert per incident until 3 seconds pass without a risky person.

3.5 Usability and Maintainability

- Responsive UI (Tailwind CSS) for control-center screens and mobile devices; Hebrew interface for the operators.
- Clean layered code: routes → middleware → services → models, with one file per responsibility.
- Automated tests: an end-to-end backend suite and unit tests for the AI rules (see section 9).

4. System Architecture and Design

4.1 Architecture Overview

SafeCross is built as a three-tier web system plus an external AI service:

- **Client tier** – React & Vite SPA with role-based dashboards. Communicates with the server over REST (Axios / fetch) and listens to Socket.io events.
- **Server tier** – Node.js & Express REST API and Socket.io server. Requests pass through routes → authentication (JWT or API key) → role authorization → services → Mongoose models.
- **Database tier** – MongoDB Atlas with five collections: users, crosswalks, cameras, leds, alerts.
- **External services** – Python AI hazard detection (YOLOv8), Cloudinary (incident images), OpenStreetMap / Nominatim (maps and geocoding, called from the frontend).

SafeCross — Real-Time Smart Traffic Management System Architecture

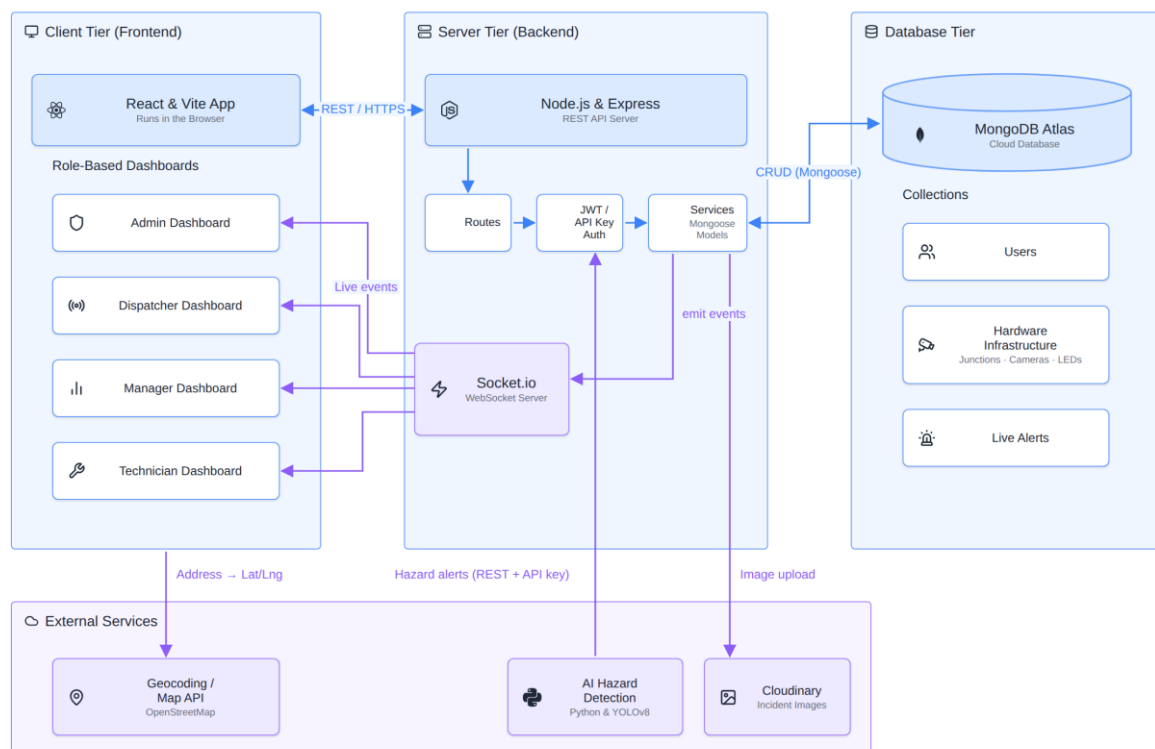


Figure 2 – System architecture and data flow

4.2 Server-Side Architecture (Backend Modules)

Module	Files	Responsibility
Entry point	index.js , app.js	app.js builds the Express app (CORS, JSON body up to 10 MB, routes). index.js creates the HTTP server, attaches Socket.io, connects to MongoDB and starts the change streams.
Routes	routes/*.js	One router per resource: alerts, crosswalks, cameras, leds, users, analytics, detect.
Middleware	middleware/*.js	authenticationMiddleware (JWT), authorizationMiddleware (roles), apiKeyMiddleware (sensor key).
Services	services/*.js	Business logic: CRUD for each resource, alert creation with image upload, analytics aggregation, user management.
Models	models/*.js	Mongoose schemas with validation (required fields, enums, references).
Config	config/db.js , socket.js , cloudinary.js	Database connection, Socket.io server, emit helpers and change streams, Cloudinary setup.
AI service	backend/ai-service/*.py	main.py (video), detector.py (YOLO), tracker.py , risk_rules.py , alert_sender.py , app.py (FastAPI /detect).

4.2.1 REST API

Method & path	Access	Description
POST /api/users/login	Public	Login → { token, role, username }
POST /api/users/register	Admin	Create a user
GET / PUT / DELETE /api/users/:id]	Admin	List, update, delete users
PATCH /api/users/:id/status	Admin	Suspend / activate a user
GET /api/crosswalks · /cameras · /leds	All roles	List all equipment (optional ?junctionId=)
POST /api/crosswalks · /cameras · /leds	Admin	Create equipment → infra_added
PUT /api/crosswalks · /cameras · /leds /:id	Admin, Technician	Update equipment → infra_updated
DELETE /api/crosswalks · /cameras · /leds /:id	Admin	Delete equipment → infra_deleted
POST /api/alerts	Sensor (x-api-key)	Create an alert (uploads snapshot to Cloudinary)
GET /api/alerts	All roles	List alerts (newest first)
PUT /api/alerts/:id	Admin, Dispatcher, Technician	Mark handled / re-open → alert_resolved / alert_reopened
GET /api/analytics/dashboard?filter=	Manager, Admin	Statistics for top5 / school / all
POST /api/detect	Sensor (x-api-key)	Forward one image to the AI service for detection

4.2.2 Real-Time Events (Socket.io)

Event	Emitted when	Listeners
newAlert	A new alert is inserted (change stream)	Dispatcher, Crosswalk details
alertUpdated	An alert document changes (change stream)	Dispatcher, Crosswalk details
alert_resolved / alert_reopened	A dispatcher handles / re-opens an alert	Manager (re-fetch statistics)
infra_added / infra_updated / infra_deleted	Equipment is created / updated / deleted (payload { type, payload })	Admin, Manager, Technician
user_added / user_updated / user_deleted	A user changes (change stream on users)	Admin

4.3 Client-Side Architecture

The client is a Single Page Application (SPA) built with React and Vite. It gives a smooth and fast user experience with no page reloads. The client side is based on the following mechanisms:

- **State management and lifecycle:** React Hooks ([useState](#), [useEffect](#)) manage the local data of each screen. The connection to the Socket.io server delivers real-time events that update the component state directly, so React re-renders only the affected part of the page.
- **Role-based navigation:** after login, the server returns a JWT and the user's role. The client stores the token in [localStorage](#) and React Router sends the user to the matching dashboard. On logout the token is removed. Access control itself is enforced by the server, which rejects any request without a valid token or with the wrong role (401 / 403).
- **Secure communication with the server:** every REST call (Fetch API) carries the JWT in the **Authorization: Bearer** header. The server address is read from an environment variable ([VITE_API_URL](#)), so the same code runs locally and in production.
- **Page structure and responsive design:** each role has its own dashboard page. The styling uses Tailwind CSS with responsive breakpoints, so the interface fits large control-center screens as well as mobile devices for field technicians. Maps are displayed with React-Leaflet and charts with Recharts.

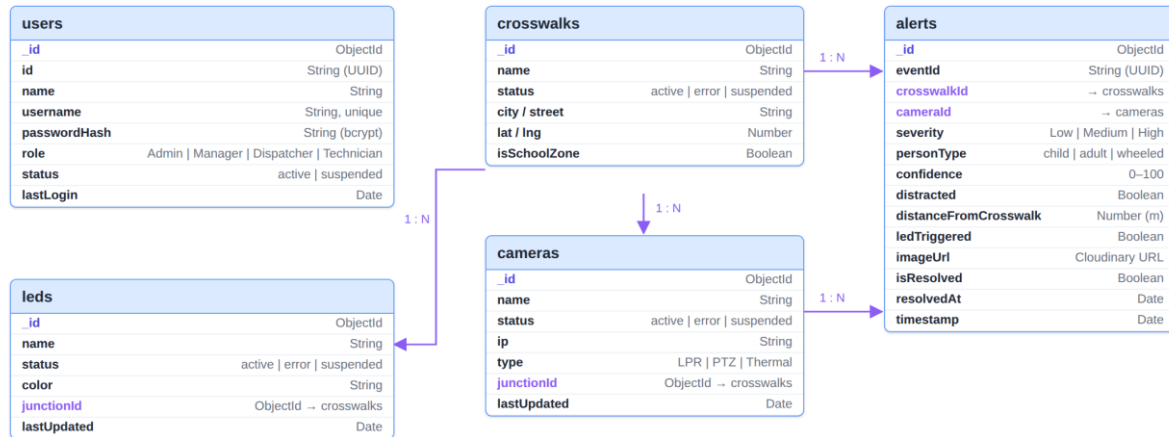
4.4 AI Service Pipeline

[main.py](#) reads a video file or camera stream and runs, for every analyzed frame: **detector** (YOLOv8 → persons, vehicles, bicycles/motorcycles, phones) → **tracker** (stable ID per person) → **risk rules** (kinematics and 17 cases) → **alert sender** (background queue → [POST /api/alerts](#)). A FastAPI app ([app.py](#)) also exposes [POST /detect](#) for single-image detection, used by the backend [/api/detect](#) bridge.

5. Data Structure (Database)

The database is MongoDB Atlas (document-oriented NoSQL). Each collection is defined by a Mongoose schema that enforces types, required fields and allowed values (enums). Cameras and LED strips reference their junction by ObjectId (`junctionId`); alerts store the junction and camera ids.

SafeCross — MongoDB Collections (Mongoose schemas)



Arrows point from the referenced collection (1) to the collection holding the reference (N). Users are independent (role-based access).

Figure 3 – Collections and relations

5.1 Collections

Collection	Purpose	Key fields
users	System accounts and permissions	username (unique), passwordHash (bcrypt), role, status, lastLogin
crosswalks	Monitored junctions	name, status, city, street, lat, lng, isSchoolZone
cameras	Cameras at a junction	name, status, ip, type, junctionId
leds	LED warning strips at a junction	name, status, color, junctionId
alerts	Hazard events from the AI	crosswalkId, cameraId, severity, personType, confidence, distracted, distanceFromCrosswalk, approachSpeed, ledTriggered, imageUrl, isResolved, resolvedAt, timestamp

The data type of each field appears in the collections diagram above (Figure 3).

Equipment status uses one shared list – `active | error | suspended` – so the technician screen can treat all hardware the same way.

6. Scientific Basis, Technologies and Algorithms

The project applies material from courses in algorithms, data structures, computer vision / machine learning, databases, data science, computer networks and information security.

6.1 Object Detection – Convolutional Neural Networks (YOLOv8)

Object detection is solved with **YOLOv8** (Ultralytics), a single-stage convolutional neural network. Unlike two-stage detectors (e.g. Faster R-CNN), YOLO predicts bounding boxes and class probabilities for the whole image in one forward pass, which makes it suitable for real-time video. The network is pre-trained on the COCO dataset; SafeCross keeps only the relevant classes: person (0), bicycle (1), car (2), motorcycle (3), bus (5), truck (7) and cell phone (67).

- **Confidence threshold** – detections below 0.5 are discarded to reduce false positives.
- **Non-maximum suppression (NMS)** and an additional IoU check (Intersection over Union > 0.7) remove duplicate boxes of the same person: $\text{IoU}(A,B) = \text{area}(A \cap B) / \text{area}(A \cup B)$.
- **Model choice** – `yolo8n` (nano) gives the best speed/accuracy trade-off on a CPU; it can be replaced with a larger model for higher accuracy.

6.2 Multi-Object Tracking

A detector sees each frame independently, but behaviour (speed, direction, stopping) can only be measured over time. The tracker links detections into tracks with a stable ID. Two interchangeable implementations exist:

- **SimpleTracker** (default) – a greedy nearest-neighbour assignment with gating: a detection is matched to an existing track only if its movement per second is below a plausible limit (6 body heights per second, at least 20 px) and its box height ratio stays within 0.6–1.7. Tracks that are missing for more than 0.5 s are closed.
- **ByteTrack** (Ultralytics) – an association algorithm that also uses low-confidence detections to keep tracks alive through occlusions.

Phones are assigned to the nearest person when the phone centre is close to the person's chest point; a person is "distracted" if a phone is seen in at least 50% of the recent frames.

6.3 Geometry – Distance to the Curb

The crosswalk zone is a polygon defined in normalized coordinates, so the same configuration works at any resolution. For each person, the feet point (bottom-centre of the box) is tested with a **point-in-polygon** algorithm (`cv2.pointPolygonTest`), and the **point-to-segment distance** to the curb edges is computed. The result is a signed distance: positive on the sidewalk, zero or negative inside the crosswalk zone.

To remove the effect of perspective (people far from the camera look smaller), all distances and speeds are normalized by the person's own body height h . A child is identified when h is below 0.9 of the reference adult height at the same image position.

6.4 Kinematic Risk Classification (Rule-Based Expert System)

For each track, the service computes kinematic features over a sliding window of 2 seconds: distance to the edge, approach speed (the change of distance over time), lateral speed, total speed, net movement, time-to-edge (distance $\div$ approach speed) and recent speed over the last 0.5 s (so a person who stops is no longer an alert). These features are matched against 17 rules; when several rules match, a priority list chooses the most severe one.

Level	Cases (examples)	Action
Low	L1 moving parallel to the road · L2 standing far from the edge · L3 moving away · L4 waiting quietly at the edge · M1 normal approach · M3 group approaching · M4 standing in the crossing zone	Logged only
Medium	M2 approaching while holding a phone · M5 brisk approach without slowing · H2 child approaching the edge · MW wheeled vehicle slowing but not enough	Early warning, LED flag
High	H1 bursting toward the road · H3 distracted and not stopping · H4 sudden appearance near the edge · H5 entering without stopping · H6 child running toward the edge · HW wheeled vehicle approaching fast	Immediate danger, LED flag

The full list of the 17 rules and their conditions appears in [Appendix A](#).

The **confidence score** of an alert combines the detector confidence with the amount of evidence: $\text{confidence} = 100 \times \text{mean detection confidence} \times \min(1, \text{frames observed} / 3)$. A person seen in only one or two frames therefore receives a lower score. An alert is sent only after the person was seen in at least 3 analysed frames, and one alert is sent per incident (an incident ends after 3 seconds with no risky person).

6.5 Data Science – Server-Side Aggregation

The manager statistics are computed inside MongoDB with an **aggregation pipeline**. First the alerts are filtered (`$match`: only handled alerts of the selected junctions); then a single `$facet` stage computes four independent results in one pass over the data:

- **KPIs** – count, number of High alerts and the average response time, `avg(resolvedAt - timestamp)` in minutes.
- **Per junction** – `$group` by junction with conditional sums for children, adults and vehicles.
- **Weekly trend** – the last 7 days (computed in Israel time with `$dateTrunc`) grouped by `$dayOfWeek`, returned as exactly 7 values Sunday–Saturday (missing days are filled with 0).
- **Severity distribution** – `$group` by severity.

The **Top-5** filter is a top-k query: group handled alerts by junction, sort by count (descending) and take the first five. The correctness of all four results is verified in the tests by an independent re-calculation in JavaScript.

6.6 Information Security

- **Password hashing** – bcrypt, an adaptive, salted hash function (cost factor 10) that is slow by design against brute-force attacks.
- **JWT** – the token payload (user id, role) is signed with HMAC-SHA256; any change to the token invalidates the signature.
- **Constant-time comparison** of the API key (`crypto.timingSafeEqual`) prevents timing side-channel attacks.

6.7 Event-Driven and Real-Time Communication

SafeCross follows the **publish/subscribe** pattern. The database is the single source of truth: MongoDB change streams (which require a replica set, as provided by Atlas) notify the server of every change, and Socket.io broadcasts the event over a persistent WebSocket connection to every subscribed client. This removes polling, lowers latency and keeps all screens consistent.

6.8 Technology Stack

Layer	Technologies
Frontend	React 19 + Vite, React Router, Tailwind CSS, Axios, Socket.io client, Recharts (charts), Leaflet / React-Leaflet (maps)
Backend	Node.js, Express 5, Socket.io, Mongoose, jsonwebtoken, bcrypt, Cloudinary SDK, dotenv
Database & storage	MongoDB Atlas, Cloudinary
AI service	Python, Ultralytics YOLOv8, OpenCV, NumPy, FastAPI, Requests
External services	OpenStreetMap tiles, Nominatim geocoding
Tools	Git & GitHub, VS Code, Postman, Node test runner, mongodb-memory-server, Python unittest

7. User Interface (UI)

The interface is in Hebrew, responsive, and uses one consistent layout: a side navigation bar with the SafeCross brand and a main working area. Each role sees only its own dashboard.

7.1 Admin Dashboard – Users and Hardware Infrastructure

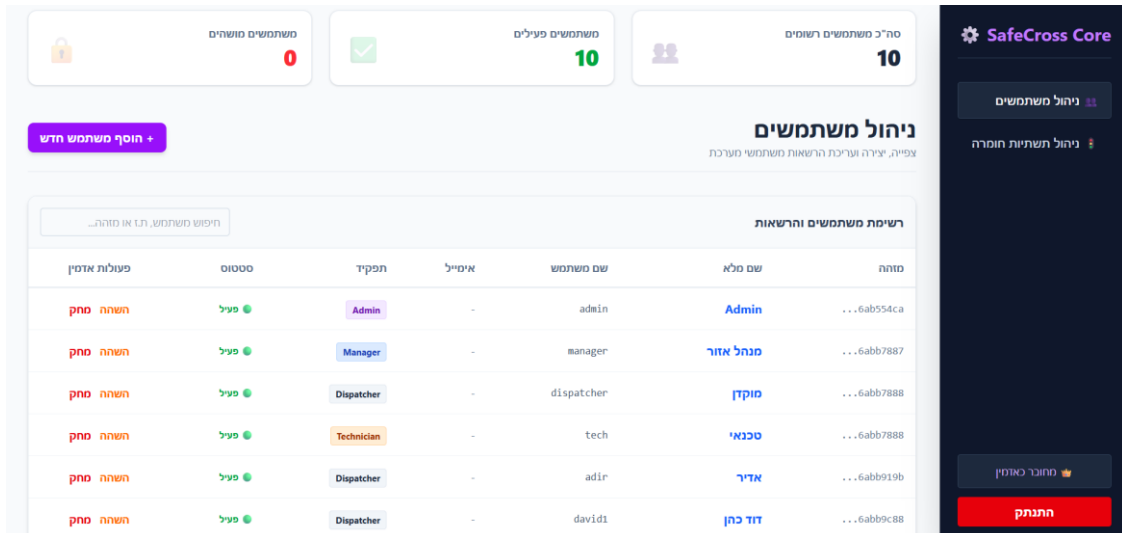


Figure 4 – Admin dashboard

Two management areas: user management (create, edit role, suspend, delete) and hardware infrastructure (junctions, cameras, LEDs). When a junction address is entered, its coordinates are filled automatically.

7.2 Manager Dashboard – Safety Analytics

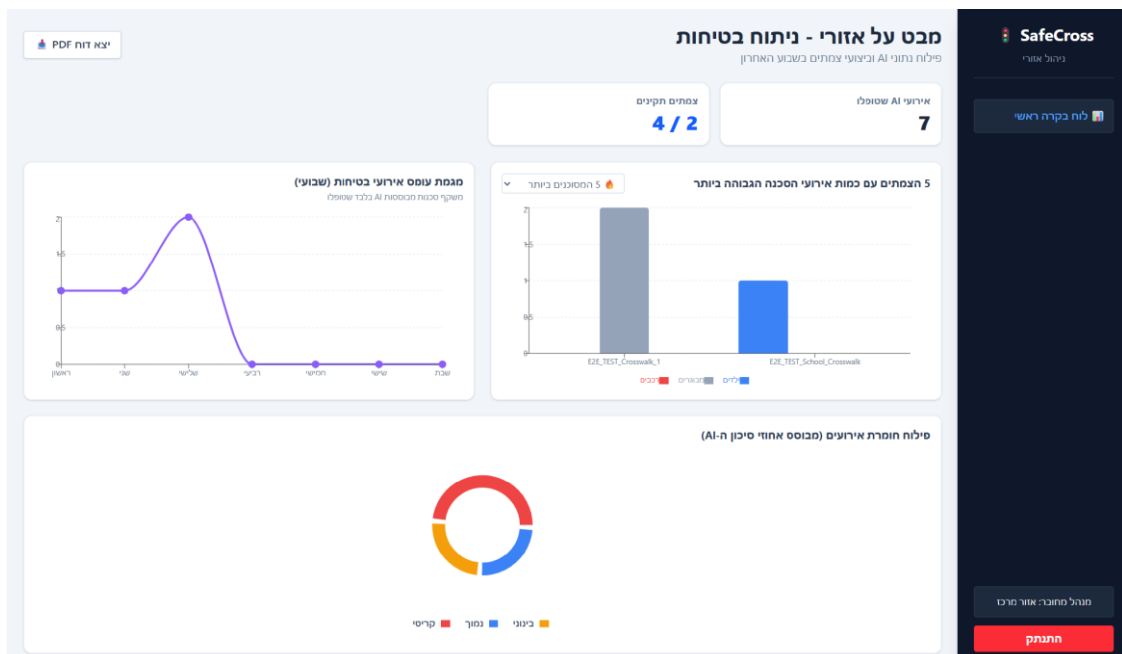


Figure 5 – Manager dashboard

KPI cards, a bar chart per junction with a Top 5 / School zones / All filter, a weekly line chart and a severity pie chart. An "Export PDF report" button at the top of the screen lets the manager save or print the current view.

7.3 Dispatcher Dashboard – Real-Time Control Center

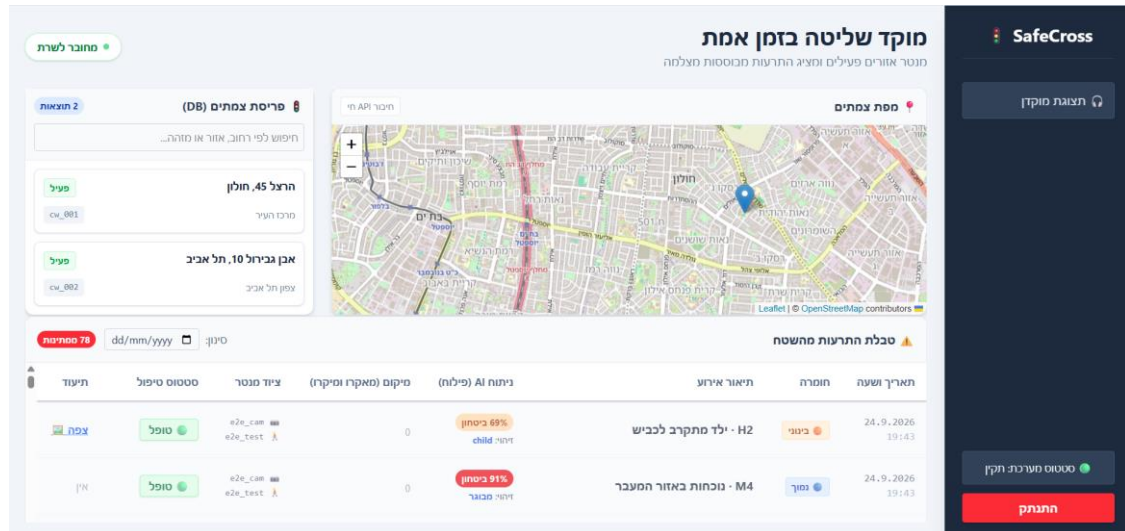


Figure 6 – Dispatcher dashboard

An interactive map of the junctions and a live alert table with severity badges and a “handled / not handled” toggle.

7.4 Technician Dashboard – Field Maintenance

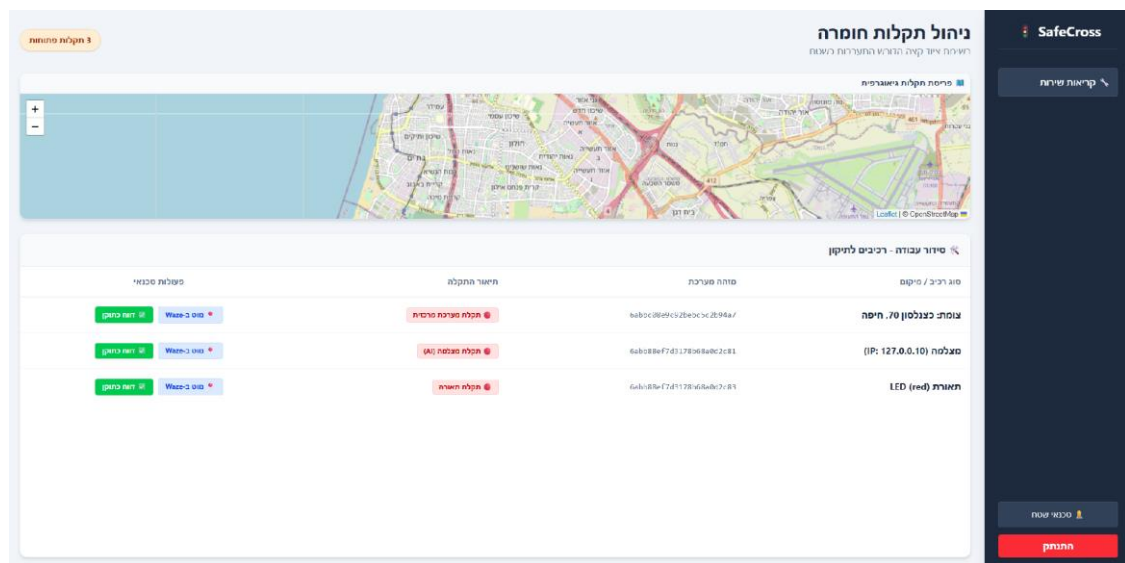


Figure 7 – Technician dashboard

A list of faulty equipment by junction, with an action to mark each item as fixed.

8. Work Flow

The full lifecycle of a hazard alert, from the camera to the management statistics:

SafeCross — Hazard Alert Lifecycle

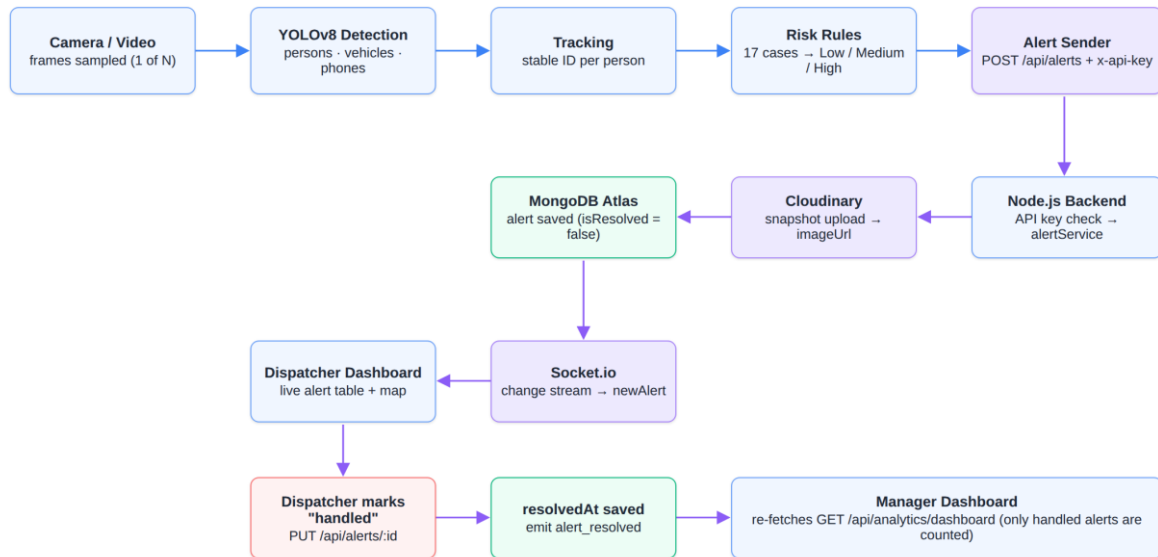


Figure 8 – Hazard alert lifecycle

1. The AI service samples frames, detects objects with YOLOv8 and tracks every person.
2. The risk rules classify the behavior; a Medium/High case creates an alert with `ledTriggered = true`.
3. The alert is sent to `POST /api/alerts` with the API key; the backend uploads the snapshot to Cloudinary and saves the alert.
4. The change stream pushes `newAlert` to all dispatchers; the alert appears on the table and the map.
5. A dispatcher marks the alert as handled; the server stores `resolvedAt` and emits `alertUpdated` and `alert_resolved`.
6. The manager dashboard re-fetches the statistics, now including the handled alert.

9. Testing and Validation

9.1 Backend End-to-End Tests

An automated end-to-end suite (`backend/tests/e2e.test.js`, Node test runner) starts a real in-memory MongoDB replica set, runs the real seed script, starts the real Express + Socket.io server on a random port and logs in with real users. Nothing is mocked. The 24 tests cover:

- **Seed and login** – migration of old data, idempotent seeding, login of the four default users with the correct role.
- **Admin infrastructure** – GET of all equipment, create / update with validation (400), permissions (401 / 403), unknown id (404), and the `infra_added` / `infra_updated` socket events with the correct type.
- **Sensor API key** – requests without a key or with a wrong key are rejected (401); a valid key saves the alert and emits `newAlert`.
- **Alerts** – `resolvedAt` is set by the server, the client value is ignored, and `alert_resolved` is emitted exactly once per resolution.
- **Manager analytics** – the response shape (7 fixed days, 3 severity labels) and each filter (all / school / top5) are compared with an independent JavaScript re-calculation from the database; unresolved alerts are not counted.

Result: **24 / 24 tests passed.**

9.2 AI Service Unit Tests

Python unit tests (`backend/ai-service/tests`) validate the risk rules (41 tests), the tracker (11 tests) and the video processor (7 tests) with synthetic tracks and generated video, including edge cases such as truncated boxes at the frame border, groups, phones and wheeled vehicles. All tests pass.

9.3 Manual and Integration Testing

- All REST endpoints were verified with Postman during development (status codes, permissions and response bodies).
- The full flow was tested with the sample videos: video → AI alerts → dispatcher screen → “handled” → manager statistics.
- Multi-client synchronization was verified by opening several browsers with different roles at the same time.

10. Solution Overview and Implementation

The project was managed with an Agile methodology in monthly sprints, coordinated through GitHub (separate backend and frontend branches merged into main) and regular team meetings.

Sprint	Focus	Main deliverables
1	Architecture & planning	Requirements, data structures, system architecture, development environments (Node.js, Express, MongoDB Atlas, React with Vite, Tailwind CSS).
2	API, database & security	Mongoose schemas, REST routes, JWT authentication and role middleware, dummy data for parallel frontend development, Postman testing.
3	AI integration & UI	Python environment with YOLOv8, detection of pedestrians and vehicles in the crosswalk zone, role-based dashboards.
4	Real-time sync & cloud	Video tracking and 17 risk cases, Socket.io live synchronization, Cloudinary images, server-side analytics, sensor API key, end-to-end tests.

10.1 Main Challenges and Solutions

10.1.1 Evolving requirements

The challenge: The requirements changed while we were building the system. The frontend team defined new data fields, events and permissions for each dashboard, and the existing backend structure no longer matched. For example, the Admin page needed a different structure for junctions, cameras and LEDs than the one built in Sprint 2.

The solution: We worked in agile iterations, one dashboard at a time, with frequent alignment meetings between the frontend and backend teams. Every API contract was agreed in writing before coding: the fields, the socket events and the permissions for each role.

10.1.2 Multi-role real-time synchronization

The challenge: Four different roles work on the same data at the same time, and each one needs to see different changes immediately. Without a clear design, screens showed outdated data, or two dispatchers handled the same alert.

The solution: We made the database the single source of truth. MongoDB change streams and server emits broadcast every change through Socket.io, after we mapped which role needs which event: `newAlert` and `alertUpdated` for dispatchers, `alert_resolved` for managers, and `infra_*` and `user_*` events for admins and technicians.

10.1.3 Reliable statistics

The challenge: The AI can produce false alarms. If every alert went straight into the manager's graphs, the statistics would be misleading and decisions would be based on wrong data.

The solution: Only alerts that a dispatcher has verified and marked as handled are counted. All calculations run on the server in a single MongoDB aggregation, so every manager sees the same numbers, and the server itself records the handling time used for the average response time.

10.1.4 Camera-independent AI rules

The challenge: A person far from the camera looks small and seems to move slowly, while a person close to it looks large and fast. Rules written in pixels worked for one video and failed for another camera angle or resolution.

The solution: All distances and speeds are normalized by the person's own body height, and the crosswalk zone is defined as a polygon in normalized coordinates (0–1). The same 17 rules therefore work for any camera position and resolution.

10.1.5 Schema expansion without breaking data

The challenge: When the data structure changed, the database already held documents in the old format. Old and new documents together caused errors in the API and wrong results in the screens.

The solution: The seed script includes a one-time migration: it detects old-format documents, clears them and inserts the demo data in the new format. The end-to-end tests start from old-format data and verify that the migration works and that running it again changes nothing.

11. Recommendations for Further Development

- **Physical LED / warning sign control** – connect the `ledTriggered` flag to a hardware controller (e.g. MQTT or a REST endpoint on the sign) to warn drivers in real time.
- **Learning from operators** – use the dispatchers’ “handled” and false-positive decisions as labelled data to fine-tune the model and the rule thresholds.
- **Live camera streams** – move from video files to RTSP streams with GPU inference and one AI worker per camera.
- **Automatic school-zone detection** – mark junctions within 300 m of an educational institution automatically, using a geospatial index on the coordinates.
- **Audit log and reports** – record every action and generate periodic reports per junction for municipalities.
- **Smart-city integration** – integrate with municipal traffic-light controllers and expand to more cities.
- **Cloud deployment and monitoring** – deploy the backend and AI service with containers, add health monitoring and rate limiting.
- **Dedicated mobile app** for technicians with navigation and push notifications.

Links

GitHub repository: [SafeCross](#)

YouTube demo video: [YouTube link – to be added]

Appendix A: [Appendix A Risk Rules EN.pdf](#)